Fondements de la programmation

Corrigé du partiel 1

Question A. Inférences de types. Le premier terme `1 + "2"` n'est pas typable car l'inférence bloque, il est impossible d'assigner le type num à str[2] comme le voudrait la règle plus.

$$\dfrac{\text{num}\ \dfrac{}{\vdash \text{num}[1] : \text{num}} \qquad \boxed{\text{⚠ str} \neq \text{num}}\ \dfrac{}{\vdash \text{str}[2] : \text{num}}}{\vdash \text{plus}(\text{num}[1], \text{str}[2]) : \text{num}}\ +$$

Le deuxième terme `if 3 >= 0 then "hi" else fun (x:num) => x` n'est pas typable car str $\neq$ num $\to$ num.

$$\dfrac{\text{num}\ \dfrac{}{\vdash \text{num}[3] : \text{num}} \qquad \text{str}\ \dfrac{}{\vdash \text{str}[\text{hi}] : \boxed{\tau = \text{str}}} \qquad \dfrac{\dfrac{}{x : \text{num} \vdash x : \text{num}}\ \text{var}}{\lambda\{\text{num}\}(x.x)) : \boxed{\tau = \text{num} \to \text{num}}}\ \text{abs}}{\vdash \text{ifpos}(\text{num}[3], \text{str}[\text{hi}], \lambda\{\text{num}\}(x.x)) : \tau \quad \boxed{\text{⚠ str} \neq \text{num} \to \text{num}}}\ \text{ifpos}$$

Le troisième terme `let x = (22, 23) in Some(x) as {Some: num | None: Unit}` n'est pas typable car prod(num, num) $\neq$ num. Posons $T = \text{sum}(\text{Some}\,.\,\text{num}; \text{None}\,.\,\text{prod}())$, l'inférence bloque.

$$\dfrac{\text{prod}\ \dfrac{\text{num}\ \dfrac{}{\vdash \text{num}[22] : \text{num}} \qquad \dfrac{}{\vdash \text{num}[23] : \text{num}}\ \text{num}}{\vdash \text{tuple}(\text{num}[22], \text{num}[23]) : \text{prod}(\text{num}, \text{num})} \qquad \dfrac{\boxed{\text{⚠ prod}(\text{num}, \text{num}) \neq \text{num}}}{x : \text{prod}(\text{num}, \text{num}) \vdash x : \text{num}}}{\Gamma \vdash \text{let}(\text{tuple}(\text{num}[22], \text{num}[23]), x.\,\text{Some}\{T\}(x)) : T}\ \begin{matrix}\text{inj}\\ \text{let}\end{matrix}$$

Le quatrième terme est typable `fun (f: str -> str) => fun (g: str -> num) => fun (x:str) => g(f(x))` de type (str $\to$ str) $\to$ (str $\to$ num) $\to$ str $\to$ num. On pose $\Gamma = f : \text{str} \to \text{str}, g : \text{str} \to \text{num}, x : \text{str}$ et en utilisant une notation λ pour rester compact.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\text{id}\ \dfrac{}{\Gamma \vdash g : \text{str} \to \text{num}} \qquad \begin{matrix}\text{id}\ \dfrac{}{\Gamma \vdash f : \text{str} \to \text{str}} \qquad \dfrac{}{\Gamma \vdash x : \text{str}}\ \text{id}\\ \text{ap}\end{matrix}\ \dfrac{}{\Gamma \vdash f(x) : \text{str}}}{\Gamma \vdash g(f(x)) : \text{num}}\ \text{ap}}{f : \text{str} \to \text{str}, g : \text{str} \to \text{num} \vdash \lambda(x : \text{str}) g(f(x)) : \text{str} \to \text{num}}\ \text{abs}}{f : \text{str} \to \text{str} \vdash \lambda(g : \text{str} \to \text{num}) \lambda(x : \text{str}) g(f(x)) : (\text{str} \to \text{num}) \to \text{str} \to \text{num}}\ \text{abs}}{\lambda(f : \text{str} \to \text{str}) \lambda(g : \text{str} \to \text{num}) \lambda(x : \text{str}) g(f(x))) : (\text{str} \to \text{str}) \to (\text{str} \to \text{num}) \to \text{str} \to \text{num}}\ \text{abs}$$

Le cinquième terme `(fun (x: num -> num) => x)(fun (x: num) => x)` est typable de type num $\to$ num.

$$\dfrac{\begin{matrix}\text{abs}\\ \text{ap}\end{matrix}\ \dfrac{\dfrac{\text{id}\ \dfrac{}{x : \text{num} \to \text{num} \vdash x : \text{num} \to \text{num}}}{\vdash \lambda(x : \text{num} \to \text{num}) x : (\text{num} \to \text{num}) \to (\text{num} \to \text{num})} \qquad \dfrac{\dfrac{}{x : \text{num} \vdash x : \text{num}}\ \text{id}}{\vdash \lambda(x : \text{num}) x : \text{num} \to \text{num}}\ \text{abs}}{\vdash (\lambda(x : \text{num} \to \text{num}) x)(\lambda(x : \text{num}) x) : \text{num} \to \text{num}}$$

Question B. Filtrage de motifs. Soit T = `{Nord: Unit | Sud: Unit | Est: Unit | Ouest: Unit`. Considérons une fonction (une lambda-expression) qui prend en entrée une expression de type T et filtre en oubliant le cas `Nord` :

$$e = fun(x : T) \Rightarrow \text{match}(x; \text{Est}(x_1).\,\text{str}[est]; \text{Ouest}(x_2).\,\text{str}[ouest]; \text{Sud}(x_3).\,\text{str}[sud])$$

Essayons de typer cette expression.

$$\text{abs}\ \frac{\dfrac{\dfrac{\boxed{\text{⚠}\ T \neq \text{sum}(\text{Est}\,.\tau_1; \text{Ouest}\,.\tau_2; \text{Sud}\,.\tau_3)}}{x : T \vdash x : \text{sum}(\text{Est}\,.\tau_1; \text{Ouest}\,.\tau_2; \text{Sud}\,.\tau_3)} \quad \text{str}\ \dfrac{}{x : T, x_1 : \tau_1 \vdash \texttt{"est"} : \text{str}} \quad \ldots}{x : T \vdash \underbrace{\text{match}(x; \text{Est}(x_1).\texttt{"est"}; \text{Ouest}(x_2).\texttt{"ouest"}; \text{Sud}(x_3).\texttt{"sud"})}_{\texttt{match x with case Est(x1) => "est" | ...}} : \tau = \text{str}}\ \text{match}}{\vdash \lambda\{T\}(x.\,\text{match}(x; \text{Est}(x_1).\texttt{"est"}; \text{Ouest}(x_2).\texttt{"ouest"}; \text{Sud}(x_3).\texttt{"sud"})) : T \to \tau}$$

Il n'est pas possible d'unifier le type T et le type $\text{sum}(\text{Est}\,.\tau_1; \text{Ouest}\,.\tau_2; \text{Sud}\,.\tau_3)$ puisque T est une somme d'arité 4 et le second type est une somme d'arité 3.

Le langage EF rejette au moins certains cas de pattern matching incomplets.

Question C. Map. Notons e le terme à réduire. Soient :

e_1=`match x with Some(x1) => Some(f(x1)) as T | None(x2) => None(unit) as T)`

e_2=`fun (f: num -> num) => fun (x: T) =>` e_1

e_3=`fun (y:num) => y + 1`

e_4=`None(unit) as T`

On a $e = (e_2(e_3))(e_4)$. e_2 et e_3 sont des abstractions donc des valeurs, par contre $e_2(e_3)$ n'est pas une valeur. $e_2(e_3) \mapsto [e_3/f]\lambda(x : T)e_1 = \lambda(x : T)[e_3/f]e_1$ qui est une abstraction donc une valeur. Ainsi $e \mapsto (\lambda(x : T)[e_3/f]e_1)(e_4)$ et comme e_4 est une valeur (car `unit` le produit vide est une valeur) on a encore $(\lambda(x : T)[e_3/f]e_1)(e_4) \mapsto [e_4/x][e_3/f]e_1$. Appelons e' cette dernière expression et explicitons le résultat des substitutions. On trouve :

$$e' = \text{match}(e_4; \text{Some}(x1).\,\text{Some}\{T\}((\lambda(y : \text{num})\texttt{y + 1})(x1)); \text{None}(x2).\,\text{None}\{T\}(\texttt{unit}))$$

Comme $e_4 = \text{None}\{\text{T}\}(\texttt{unit})$ est une valeur, $e' \mapsto [\texttt{unit}/x_2]\,\text{None}\{T\}(\texttt{unit}) = \text{None}\{T\}(\texttt{unit})$ qui est une valeur ce qui achève la réduction.

Question D. Langage. Un langage de programmation est dit avoir des fonctions de première classe lorsque les fonctions dans ce langage sont traitées comme n'importe quelle autre expression. Par exemple, dans un tel langage, une fonction peut être transmise en tant qu'argument à d'autres fonctions, peut être retournée par une autre fonction et peut être affectée en tant que valeur à une variable ou stockée dans une structure de données. En C, Pascal, Ada, Basic les fonctions ne sont pas traitées comme des citoyennes de premières classe du langage. En OCaml, Java 8+, Scala, Haskell, JavaScript, Python, Scheme, Clojure les fonctions sont des citoyennes de première classe. Strachey a introduit cette notion dans les années 60.

Question E. Démonstration (bonus). Propriété P : si un terme e est une erreur, alors il contient soit un sous-terme $\text{div}(\text{num}[n], \text{num}[0])$ soit un sous-terme error.

Par induction structurelle sur la définition des erreurs. Si la dernière règle utilisée est « let err in e » alors par hypothèse d'induction e_1 (qui s'applique car e_1 est une erreur) contient un tel sous-terme et donc e également.

$$\frac{e_1\,\text{err}}{\text{let}(e_1, x.e_2)\,\text{err}}\ \text{let err in e} \qquad \frac{e_1\,\text{val} \quad e_2\,\text{err}}{e = \text{let}(e_1, x.e_2)\,\text{err}}\ \text{let v in err}$$

Si la dernière règle est « let v in err », alors, par hypothèse d'induction e_2 contient un tel sous-terme, donc e également.

De même pour les autres cas. Les cas de base sont immédiats (error est un bien une sous-expression de error, de même pour $\text{div}(\text{num}[n], \text{num}[0])$).